CS4031 - Compiler Construction

Assignment 03

Bottom-Up Parser Design & Implementation

(SLR(1) and LR(1) Parsing)

Spring 2026

Important Information

Deadline: As per GCR

Submission: RollNumber1-RollNumber2-Section.zip (e.g., 22i1234-22i5678-A.zip)

Language: C, C++, or Java (choose one)

---

1 Instructions

1. Work in the same pairs as Assignment 2 (groups cannot be changed)
2. Must be implemented in C, C++, or Java (choose one language for entire assignment)
3. Code must be generic and reusable
4. No built-in compiler libraries allowed
5. Proper indentation, comments, and presentable output required
6. Plagiarism = ZERO marks
7. Submit only .ZIP files on Google Classroom
8. No email submissions accepted
9. Test thoroughly before submission

2 Assignment Overview

This assignment focuses on Bottom-Up Parser Design and Implementation. You will build complete SLR(1) and LR(1) parsers that:

1. Read a Context-Free Grammar (CFG) from a file
2. Augment the grammar with a new start symbol
3. Compute LR(0) items and construct the Canonical Collection of LR(0) items
4. Build SLR(1) parsing table using FOLLOW sets
5. Construct the Canonical Collection of LR(1) items (with lookaheads)

6. Build LR(1) parsing table using these LR(1) items
7. Parse input strings using shift-reduce parsing with a stack
8. Handle shift/reduce and reduce/reduce conflicts
9. Generate parse trees for accepted strings

3 Part 1: SLR(1) Parser Implementation

3.1 Task 1.1: Input CFG

Input Format:

- Read CFG from a text file
- One production per line
- Format: NonTerminal -> production1 | production2 | ...
- Use -> as arrow symbol
- Use | to separate alternatives
- Terminals: lowercase letters, operators, keywords
- Non-terminals: Multi-character names starting with uppercase (e.g., Expr, Term, Factor)
- Single-character non-terminals (E, T, F, etc.) are NOT allowed
- Epsilon: use epsilon or @

Example Input File (grammar.txt):

Expr -> Expr + Term | Term Term -> Term * Factor | Factor Factor -> (Expr) | id

3.2 Task 1.2: Grammar Augmentation

Purpose: Create a unique accepting state for the parser.

Algorithm:

- Add new start symbol S' (S-prime)
- Add production: S' -> S
- Where S is the original start symbol

Output: Display the augmented grammar.

3.3 Task 1.3: LR(0) Items Construction

Purpose: Build the foundation for parsing table construction.

LR(0) Item Format:

- An item is a production with a dot (•) indicating current position
- Example: A -> α • β where α and β are strings of grammar symbols

CLOSURE Operation:

Given a set of items I:

1. Add all items in I to CLOSURE(I)
2. For each item A -> α • B β in I where B is non-terminal:
   - Add B -> • γ for all productions B -> γ
3. Repeat step 2 until no new items can be added

GOTO Operation:

GOTO(I, X) where I is a set of items and X is a grammar symbol:

1. Find all items A -> α • X β in I
2. Move the dot past X: A -> α X • β
3. Return CLOSURE of these items

Canonical Collection Algorithm:

1. Initialize: C = {CLOSURE({S' -> • S})}
2. For each item set I in C and each grammar symbol X:
   a. Compute GOTO(I, X)
   b. If GOTO(I, X) is not empty and not in C, add it to C
3. Repeat until no new item sets can be added

Output: Display all item sets (states) with their items numbered (I0, I1, I2, ...).*Task 1.4: SLR(1) Parsing Table Construction*

Table Construction Algorithm:

For each state I:

1. **Shift Actions:**
   a. If item A -> α • a β is in I and GOTO(I, a) = Ij
   b. Set ACTION[i, a] = shift j
2. **Reduce Actions:**
   a. If item A -> α • is in I (dot at end)
   b. For each terminal a in FOLLOW(A):
      - Set ACTION[i, a] = reduce A -> α
3. **Accept Action:**
   a. If item S' -> S • is in I
   b. Set ACTION[i, $] = accept

4. **GOTO Actions:**
    a. If GOTO(I, A) = Ij where A is non-terminal
    b. Set GOTO[i, A] = j

Conflict Detection:

- **Shift/Reduce Conflict:** Both shift and reduce actions for same state and terminal
- **Reduce/Reduce Conflict:** Two or more reduce actions for same state and terminal

Output:

- Display complete SLR(1) parsing table
- Rows: States (I0, I1, I2, ...)
- Columns: Terminals (ACTION) + Non-terminals (GOTO)
- Report any conflicts found
- Indicate if grammar is SLR(1) or not

3.4 Task 1.5: SLR(1) Parsing Algorithm

Stack-Based Parsing:

1. Initialize: Stack = [0], Input with $ appended
2. Let s = top of stack, a = current input symbol
3. Repeat until accept or error:
    a. If ACTION[s, a] = shift t:
        i. Push a, then push t
        ii. Advance input pointer
    b. If ACTION[s, a] = reduce A -> β:
        i. Pop 2 × |β| symbols from stack
        ii. Let t = top of stack after popping
        iii. Push A, then push GOTO[t, A]
    c. If ACTION[s, a] = accept:
        i. Parsing successful
    d. If ACTION[s, a] is empty:
        i. Report error and halt

Display at Each Step:

- Step number
- Current stack contents (symbols and states)
- Remaining input string
- Action taken (shift, reduce, accept, error)

4 Part 2: LR(1) Parser Implementation

4.1 Task 2.1: LR(1) Items Construction

Purpose: Build more powerful parser that uses lookahead information.

LR(1) Item Format:

- An LR(1) item is [A -> α • β, a]
- Where a is a lookahead terminal (what can follow A)
- The lookahead a restricts when reductions can happen

CLOSURE Operation for LR(1):

Given a set of items I:

1. Add all items in I to CLOSURE(I)
2. For each item [A -> α • B β, a] in I where B is non-terminal:
    a. For each production B -> γ:
    b. For each terminal b in FIRST(βa):
    c. Add [B -> • γ, b] to CLOSURE(I)
3. Repeat until no new items can be added

GOTO Operation for LR(1):

GOTO(I, X) where I is a set of LR(1) items and X is a grammar symbol:

1. Find all items [A -> α • X β, a] in I
2. Move the dot past X: [A -> α X • β, a]
3. Return CLOSURE of these items

Canonical LR(1) Collection Algorithm:

1. Initialize: C = {CLOSURE({[S' -> • S, $]})}
2. For each item set I in C and each grammar symbol X:
    a. Compute GOTO(I, X)
    b. If GOTO(I, X) is not empty and not in C, add it to C
3. Repeat until no new item sets can be added

Output: Display all LR(1) item sets with lookaheads.

4.2 Task 2.2: LR(1) Parsing Table Construction

Table Construction Algorithm:

For each state I:

1. **Shift Actions:**
    a. If [A -> α • a β, b] is in I and GOTO(I, a) = Ij
    b. Set ACTION[i, a] = shift j
2. **Reduce Actions:**
    a. If [A -> α •, a] is in I (dot at end)
    b. Set ACTION[i, a] = reduce A -> α
    c. Note: Reduction ONLY on lookahead a (not FOLLOW(A))
3. **Accept Action:**
    a. If [S' -> S •, $] is in I
    b. Set ACTION[i, $] = accept
4. **GOTO Actions:**
    a. Same as SLR(1)

Key Difference from SLR(1):

- SLR(1) uses FOLLOW(A) for reductions (all possible lookaheads)
- LR(1) uses specific lookahead a in the item (more precise)
- This precision resolves conflicts that SLR(1) cannot handle

Output:

- Display complete LR(1) parsing table
- Report any conflicts (should be none for valid LR(1) grammars)
- Compare number of states with SLR(1)

4.3 Task 2.3: LR(1) Parsing Algorithm

The parsing algorithm is identical to SLR(1) - only the parsing table differs. Use the same stack-based approach described in Section 3.5.

5 Part 3: Parser Comparison & Analysis

5.1 Task 3.1: Conflict Analysis

Requirements:

- Test both parsers on the same grammars
- Identify any conflicts in SLR(1) parsing table
- Show how LR(1) resolves these conflicts (if any)
- Provide at least one grammar where LR(1) succeeds but SLR(1) has conflicts

5.2 Task 3.2: Performance Comparison

Compare:

- Number of states generated (SLR(1) vs LR(1))
- Parsing table size (memory usage)
- Time to construct parsing tables
- Parsing speed for sample inputs

5.3 Task 3.3: Parse Tree Generation

Requirements:

- Generate parse trees for successfully parsed strings
- Display trees in readable format (text-based or graphical)
- Show the reduction steps that built the tree
- Root: Start symbol, Leaves: Terminals, Internal nodes: Non-terminals

6 Example Execution

6.1 Example Grammar

Expr -> Expr + Term | Term Term -> Term * Factor | Factor Factor -> (Expr) | id

6.2 After Augmentation

ExprPrime -> Expr Expr -> Expr + Term | Term Term -> Term * Factor | Factor Factor -> (Expr) | id

6.3 Sample LR(0) Items (State I0)

I0: ExprPrime -> • Expr Expr -> • Expr + Term Expr -> • Term Term -> • Term * Factor Term -> • Factor Factor -> • (Expr) Factor -> • id

6.4 Sample SLR(1) Parsing Trace

Input: id + id * id $

Step	Stack	Input	Action
1	0	id + id * id $	Shift 5
2	0 id 5	+ id * id $	Reduce Factor -> id
3	0 Factor 3	+ id * id $	Reduce Term -> Factor
4	0 Term 2	+ id * id $	Reduce Expr -> Term
5	0 Expr 1	+ id * id $	Shift 6
...	...	...	(continues until accept)

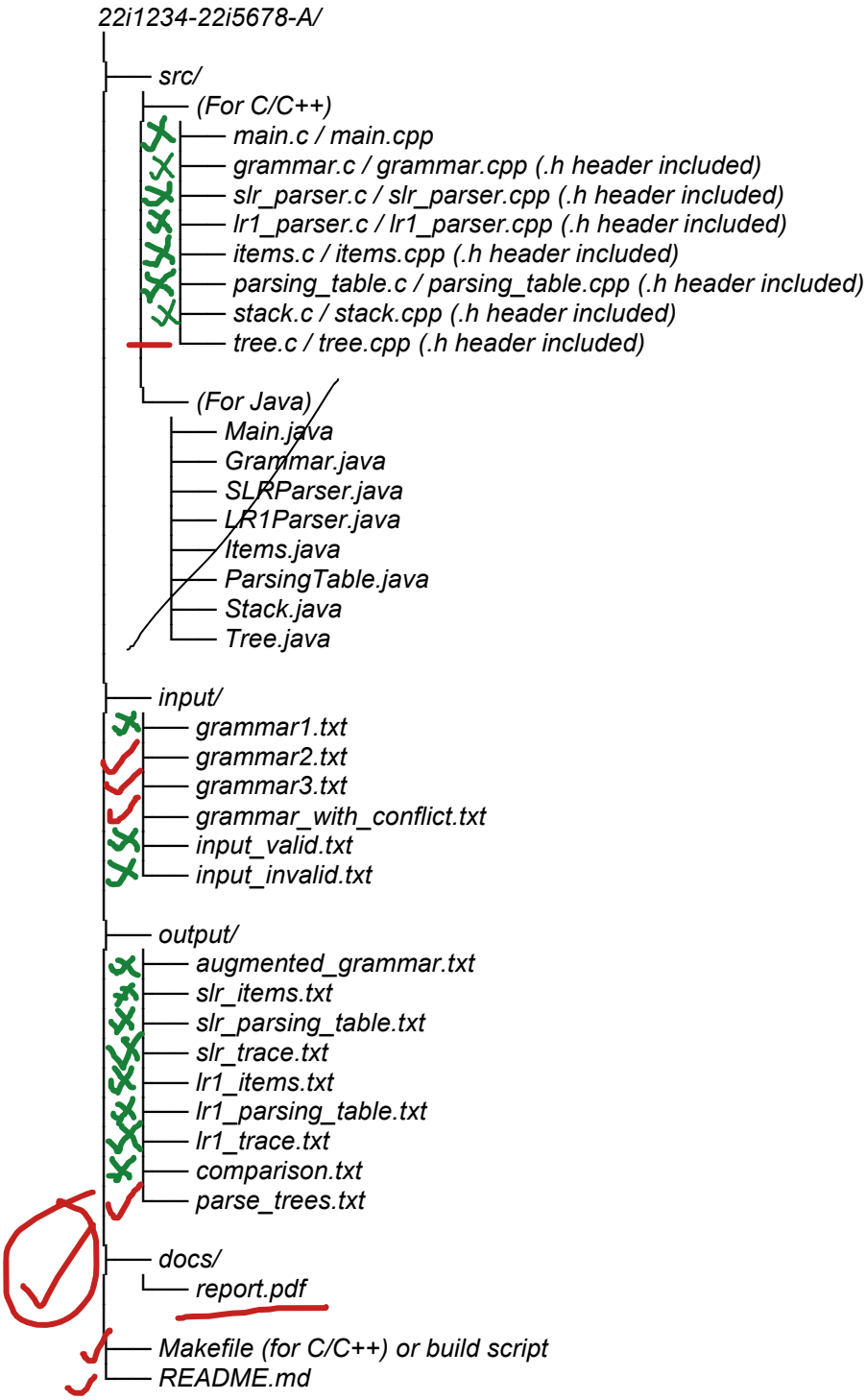
Result: String accepted successfully!

7 Deliverables

7.1 Required Files

Note: Include only files for the language you chose. Do not mix languages.

Your ZIP file must contain:



7.2 Report Requirements (report.pdf)

Include the following sections:

1. **Introduction:** Brief overview of bottom-up parsing and LR parsers
2. **Approach:**
    - Data structures used for items, item sets, and parsing tables
    - Algorithm implementation details (CLOSURE, GOTO)
    - Design decisions and trade-offs
    - How you handled lookaheads in LR(1) items
3. **Challenges:** Difficulties faced and solutions
4. **Test Cases:**
    - At least 3 different grammars tested
    - At least 5 input strings per grammar
    - Include both valid and invalid strings
    - Test case showing SLR(1) conflict resolved by LR(1)
5. **Comparison Analysis:**
    - SLR(1) vs LR(1) parsing power
    - Number of states comparison
    - Table construction time
    - Memory usage
6. **Sample Outputs:**
    - Screenshots of program execution
    - Item sets and parsing tables
    - Parse trees for accepted strings
    - Conflict examples and resolutions
7. **Conclusion:** Lessons learned and insights

7.3 README.md Requirements

- Team members with roll numbers
- Programming language used (C, C++, or Java)
- Compilation instructions (language-specific)
- Execution instructions with examples
- Input file format specification

- Grammar file format
- Sample command to run SLR(1) parser
- Sample command to run LR(1) parser
- Known limitations (if any)

8 Testing Guidelines

8.1 Test Grammar 1 (Simple Expression)

Expr -> Expr + Term | Term Term -> Factor Factor -> id

8.2 Test Grammar 2 (With Multiplication)

Expr -> Expr + Term | Term Term -> Term * Factor | Factor Factor -> (Expr)
| id

8.3 Test Grammar 3 (Classic SLR(1) Conflict Example)

Start -> L = R | R L -> * R | id R -> L

Note: This grammar is LR(1) but NOT SLR(1). Use it to demonstrate the difference.

8.4 Test Grammar 4 (Dangling Else)

Stmt -> if Expr then Stmt | if Expr then Stmt else Stmt | other Expr -> id

8.5 Test Requirements

- Valid strings that should be accepted
- Invalid strings with syntax errors
- Strings that expose parser conflicts
- Empty input
- Grammar demonstrating LR(1) superiority over SLR(1)
- Complex nested structures

9 Important Requirements

- Choose ONE language (C, C++, or Java) for the entire assignment
- Implement BOTH SLR(1) and LR(1) parsers - both are required
- Build Canonical LR(0)/LR(1) collections correctly
- Handle lookaheads properly in LR(1) items
- Generate parse trees for accepted strings
- Identify and document conflicts (if any)
- Start with Part 1 (SLR), complete it fully before moving to Part 2 (LR)
- Test each parser separately before comparison
- Use modular programming - separate files for different functionalities
- For C/C++: Handle memory properly (no memory leaks - use valgrind)
- For Java: Follow proper OOP principles and exception handling
- Test with multiple grammars (simple and complex)
- Include edge cases in testing (epsilon productions, conflicts)
- Document your code thoroughly
- Provide clear comparison between SLR(1) and LR(1) in your report

10 Common Pitfalls to Avoid

- Not augmenting the grammar before building items
- Incorrect CLOSURE computation (missing closure items)
- Not handling epsilon in FIRST sets for LR(1) lookaheads
- Confusing SLR(1) reduce actions (using FOLLOW) with LR(1) (using lookahead)
- Pushing symbols in wrong order during reduce operations
- Not detecting or reporting conflicts in parsing tables
- Parse tree not reflecting actual reduction sequence
- For C/C++: Memory leaks, pointer errors, buffer overflows
- For Java: Not closing file streams, poor exception handling
- Not comparing number of states between SLR(1) and LR(1)
- Missing the grammar that demonstrates LR(1) superiority

Good luck with your assignment!

Start early and test thoroughly!